Single Qubit Interleaved Randomized Benchmarking - Simulation ReadMe

Tanay Bali (Lab Partner: Atal Sahu)

May 2026

1 Overview

This write up explains the Interleaved Randomized Benchmarking functionality that has been added to the Python simulator. IRB extends from RB in order to isolate error contributions from specific a specific Clifford gate, G, as described in the Tripathi paper.

2 Global Parameters

Two additional global parameters were added for this functionality:

Constant	Value	Description
`K_IRB`	200	Sequences per length for IRB
`TARGET_GATE_NAME`	user chosen	Name of Clifford gate

Table 1: IRB Global Parameter Table

Based on the logic described in the paper, we chose a K_{IRB} that is twice that of K to ensure the IRB error bound is tightened, as IRB is inherently looser due to additional variance stemming from the target gate being interleaved.

3 Per-Gate Noise Model

Constant Used: `GATE_NOISE_TABLE`

Since standard RB applied a single gate-independent depolarizing error drawn from $N(\mu, \sigma)$, and is uniformly applied to each gate in sequence, IRB requires knowledge of the target gate's specific noise value in order to isolate its effect. In order to do this, a per-gate noise table was added with each gate having a specific mean, μ_g, and standard deviation, σ_g, based on the pulse types, and motivated by the theory given in the paper:

Gate	(μ_g, σ_g)	Physical Effect
I	$(0.001, 0.0002)$	Virtual gate / essentially error free
Z	$(0.001, 0.0002)$	Virtual gate
S, Sd	$(0.006, 0.001)$	$\pi/2$ pulse
H	$(0.008, 0.001)$	$\pi/2 + \pi/2$ decomposition pulse
X, Y	$(0.015, 0.002)$	π pulse (most noisy)
Others	$(0.01, 0.002)$	Multi-pulse

Table 2: Table of Gate Specific Noise Models

These values follow and are chosen based on the explanation given in Section 4.4 of the Tripathi paper.

4 IRB Sequence

Function Used: `irb_sequence(n, e_max, rho_in, target_gate`

This function implements one run of a complete IRB sequence of length n for a specific target Clifford gate. The logic of this sequence is highly similar to that of the standard RB sequence explained earlier, with the main difference being that the target gate has a noise value chosen based on the above listed table rather than the standard depolarizing error.

5 IRB Data Collection

Function Used: `collect_irb_data(n_max, k, e_max, rho_in, target_gate`

This function mirrors the previously constructed `collect_data` function for standard RB. A CSV file is created with the random gate sequences stored. Note, that since each gate sequence length n introduces an interleaved gate within each pair of gates, an n length sequence has gate columns from `gate_1` to `gate_2n` in the CSV.

6 Gate Infidelity Calculation

Functions Used: `irb_gate_infidelity(p, p_inter)`, `irb_error_bound(p, p_inter`

The IRB decay function is fit to the same exponential model as that of RB, and then the interleaved decay parameter, `p_inter` is combined using the equation from the paper:

$$r_G = (1/2)(1 - p_{inter}) \tag{1}$$

$$\sigma(r_G) = \sigma(p_{inter})/(2p) \tag{2}$$

The true gate infidelity is bounded within:

$$[r_G - E, r_G + E] \tag{3}$$

where E is given by the equation (21) in the Tripathi paper. It is also important to note that the simulator has the lower bound clamped to zero and flagged in the terminal if $r_G \leq E$, which physically represents the case when the noise level is large enough that E is comparable to r_G - a known limitation Tripathi mentions.

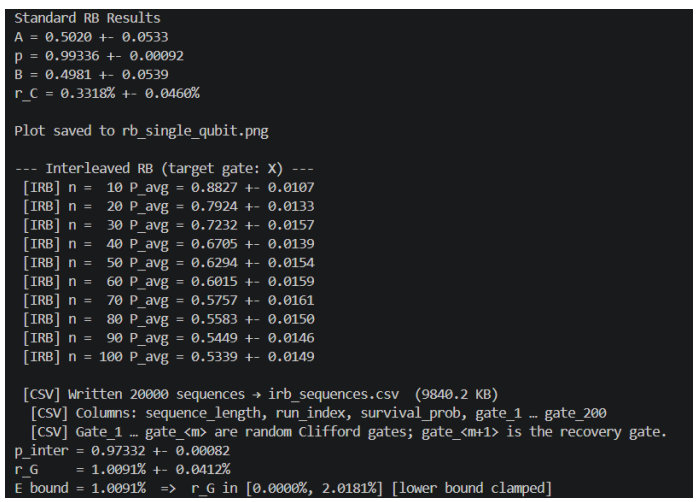
7 Visualization

Function Used: `plot_irb_single()`

The IRB output plot overlays the standard RB run results and the IRB decay results on the same axes, along with error bars and the exponential fit with numerical values provided in the legend.

8 Example Simulation Output

This simulator was tested for a max length of 100 gates, with 100 runs per length for standard RB and 200 runs per length for IRB, with the X-gate interleaved. The following terminal output screenshot and plot were produced:



```
Standard RB Results
A = 0.5020 +- 0.0533
p = 0.99336 +- 0.00092
B = 0.4981 +- 0.0539
r_C = 0.3318% +- 0.0460%

Plot saved to rb_single_qubit.png

--- Interleaved RB (target gate: X) ---
 [IRB] n =  10 P_avg = 0.8827 +- 0.0107
 [IRB] n =  20 P_avg = 0.7924 +- 0.0133
 [IRB] n =  30 P_avg = 0.7232 +- 0.0157
 [IRB] n =  40 P_avg = 0.6705 +- 0.0139
 [IRB] n =  50 P_avg = 0.6294 +- 0.0154
 [IRB] n =  60 P_avg = 0.6015 +- 0.0159
 [IRB] n =  70 P_avg = 0.5757 +- 0.0161
 [IRB] n =  80 P_avg = 0.5583 +- 0.0150
 [IRB] n =  90 P_avg = 0.5449 +- 0.0146
 [IRB] n = 100 P_avg = 0.5339 +- 0.0149

 [CSV] Written 20000 sequences → irb_sequences.csv  (9840.2 KB)
  [CSV] Columns: sequence_length, run_index, survival_prob, gate_1 … gate_200
  [CSV] Gate_1 … gate_<m> are random Clifford gates; gate_<m+1> is the recovery gate.
p_inter = 0.97332 +- 0.00082
r_G     = 1.0091% +- 0.0412%
E bound = 1.0091%  =>  r_G in [0.0000%, 2.0181%] [lower bound clamped]
```

Figure 1: Python terminal output showing standard RB + IRB data for the X-gate interleaved

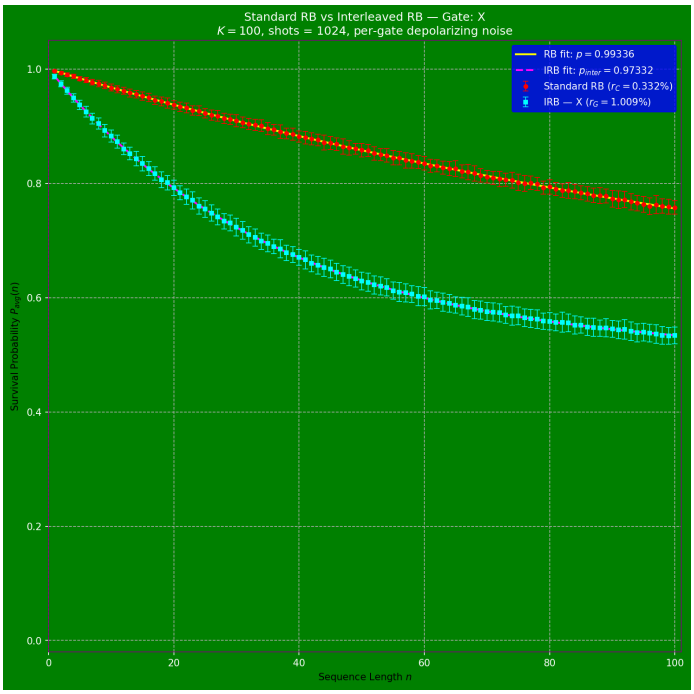


Figure 2: Overlay plot showing standard RB and IRB (X-gate) decay curves as a function of gate sequence length